🏆 IGNISIA HACKATHON · HC03 · HEALTHCARE TRACK

AURUM

Adaptive Unified Routing for Urgent Medicine

MASTER BUILD PROMPT DOCUMENT

Golden-Hour Emergency Triage & Constraint-Based Hospital Routing System

A step-by-step agent build guide covering all three user interfaces, the backend engine, the AI/ML layer, database schema, Google Maps integration, and every required feature — structured in sequential build stages for zero-hallucination code generation.

FIELD	DETAIL
Problem Statement	HC03 — Golden Hour Emergency Triage
System Name	AURUM — Adaptive Unified Routing for Urgent Medicine
Interfaces	3 — Hospital ERC · Ambulance Driver · Patient Monitor
Maps API	Google Maps Platform (Maps JS, Directions, Distance Matrix)
Primary Tech Stack	React + Node.js/FastAPI + PostgreSQL + Python ML
Document Version	1.0 — Master Agent Prompt

Document Structure & Contents

Read all stages in order. Do not skip ahead. Each stage builds on the previous. Complete the CORE stages before touching BONUS features.

STAGE	TITLE	PRIORITY
STAGE 0	Project Overview, Architecture & Setup	CRITICAL — Read First
STAGE 1	Database Schema & Seed Data	CRITICAL
STAGE 2	Backend API — Core Endpoints	CRITICAL
STAGE 3	ML Engine — APEX Severity Predictor	CRITICAL
STAGE 4	Constraint Optimizer — NEXUS Router	CRITICAL
STAGE 5	Interface A — Hospital ERC Dashboard	CRITICAL
STAGE 6	Interface B — Ambulance Driver App	CRITICAL
STAGE 7	Interface C — Patient Monitor Device UI	CRITICAL
STAGE 8	Real-Time Layer — WebSockets & Live Updates	HIGH
STAGE 9	Google Maps Integration (All Interfaces)	HIGH
STAGE 10	Mass Casualty Event (MCE) Batch Mode	HIGH
STAGE 11	Explainability Panel — SHAP Routing Rationale	HIGH
BONUS A	ORACLE — Predictive Hospital State (30-min forecast)	BONUS
BONUS B	Voice Input — Whisper ASR for EMT	BONUS
BONUS C	HELIX — Post-Incident Analytics Dashboard	BONUS
BONUS D	Pre-Arrival Activation Protocol	BONUS
BONUS E	Federated Learning Architecture Stub	BONUS

STAGE 0 — Project Overview, Architecture & Setup

0.1 — What AURUM Does (Agent Context)

AURUM is a real-time emergency triage and hospital routing system. When an ambulance picks up a patient, the EMT or the embedded patient monitor device sends vitals to AURUM. The AI engine (APEX) predicts the patient's critical care requirements. The routing engine (NEXUS) then evaluates every hospital in the region against live constraints and routes the ambulance to the best hospital — not the nearest one.

The system has three separate, purpose-built interfaces that run simultaneously and share a single backend. You will build all three interfaces in this project.

0.2 — The Three Interfaces

INTERFACE	USER	PRIMARY PURPOSE	DEVICE
Interface A — Hospital ERC	Hospital Emergency Response Centre staff	Real-time hospital stats management + incoming ambulance tracking + quick-response actions	Large desktop monitor (1920×1080+)
Interface B — Ambulance Driver App	Ambulance driver / EMT	See optimal hospital destination with live Google Map, ETA, route, and patient summary	Tablet or phone mounted in ambulance
Interface C — Patient Monitor	Auto (device-driven)	Embedded device that reads vitals from health monitoring hardware and pushes to backend; shows hospital match in real time	Embedded touchscreen in ambulance medical device

0.3 — Full System Architecture

LAYER 1 — DATA INPUT

- Patient Monitor Device (Interface C) → pushes vitals (HR, BP, SpO2, RR, GCS, temp) to POST /api/vitals via HTTP/WebSocket
- EMT manual input form (Interface B) → chief complaint, mechanism of injury, patient age/sex
- Hospital ERC Dashboard (Interface A) → staff manually updates bed counts, equipment availability, specialist on-duty status

LAYER 2 — BACKEND (Node.js + FastAPI)

- Node.js/Express REST API — auth, patient sessions, hospital data CRUD, WebSocket hub

- FastAPI (Python) — ML inference endpoints (/predict, /route, /mce-route)
- Redis — WebSocket pub/sub and real-time event bus

LAYER 3 — AI/ML ENGINE (Python)

- APEX — XGBoost severity predictor: inputs vitals → outputs care requirements (ICU/ventilator/specialist type) + survivability score
- NEXUS — PuLP integer linear programming optimizer: inputs care requirements + hospital grid → outputs optimal hospital assignment
- SHAP — Explainability layer: generates per-feature importance for every routing decision

LAYER 4 — DATABASE (PostgreSQL + PostGIS)

- patients, vitals_stream, ambulances, hospitals, hospital_resources, routing_events, mce_events tables
- PostGIS extension for geospatial distance queries

LAYER 5 — FRONTEND (React)

- Three separate React apps (or a single app with role-based routing)
- Google Maps JS API embedded in all three interfaces (size and role differs per interface)
- Socket.io client for real-time updates across all UIs

0.4 — Tech Stack Specification

LAYER	TECHNOLOGY	REASON
Frontend	React 18 + Vite + TailwindCSS	Fast dev, component isolation, utility-first styling for complex dashboards
Maps	Google Maps JavaScript API + Directions API + Distance Matrix API	User-specified; real-time routing, ETAs, traffic-aware
State Management	Zustand (lightweight) + React Query (server state)	Simple enough for hackathon, powerful enough for real-time
Backend API	Node.js + Express (REST) + Socket.io	WebSocket hub, auth middleware, hospital data CRUD
ML Service	FastAPI + Python 3.11	Async ML inference, APEX + NEXUS engine host
ML Models	XGBoost + SHAP + PuLP/OR-Tools	Gradient boosted severity prediction + ILP optimizer
Database	PostgreSQL 15 + PostGIS	Geospatial queries for proximity, relational for constraints
Cache/PubSub	Redis 7	Real-time event bus between services

Containerisation	Docker + Docker Compose	One-command setup for all services

0.5 — Project Folder Structure

Create this exact folder structure before writing any code:

```
aurum/├── frontend/│   ├── src/│   │   ├── apps/│   │   │   ├── hospital-erc/
← Interface A│   │   │   ├── ambulance-driver/   ← Interface B│   │   │   └──
patient-monitor/    ← Interface C│   │   ├── components/shared/      ← Shared
components (Map, etc.)│   │   └── hooks/                  ← Shared custom
hooks├── backend/│   ├── api/                        ← Node.js Express API│
├── ml-service/                 ← FastAPI Python ML engine│   └── db/
← Migrations + seed data└── docker-compose.yml
```

STAGE 1 — Database Schema & Seed Data

1.1 — PostgreSQL Tables

Create these tables in order. Run PostGIS extension first: CREATE EXTENSION IF NOT EXISTS postgis;

Table: hospitals

Stores all hospital records with real-time capacity state.

```
CREATE TABLE hospitals (  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
name            VARCHAR(200) NOT NULL,  address         TEXT,  lat
DECIMAL(10,7) NOT NULL,  lng             DECIMAL(10,7) NOT NULL,  location
GEOGRAPHY(POINT, 4326),  phone           VARCHAR(20),  total_icu_beds  INT DEFAULT 0,
avail_icu_beds  INT DEFAULT 0,  total_gen_beds  INT DEFAULT 0,  avail_gen_beds  INT
DEFAULT 0,  has_ventilator  BOOLEAN DEFAULT FALSE,  avail_ventilators INT DEFAULT 0,
has_trauma_centre BOOLEAN DEFAULT FALSE,  has_cath_lab    BOOLEAN DEFAULT FALSE,
has_neurosurgery BOOLEAN DEFAULT FALSE,  has_cardiac_surgery BOOLEAN DEFAULT FALSE,
has_burn_unit   BOOLEAN DEFAULT FALSE,  has_paediatrics BOOLEAN DEFAULT FALSE,
current_load_pct INT DEFAULT 0,       -- 0-100  is_active       BOOLEAN DEFAULT TRUE,
last_updated_at TIMESTAMPTZ DEFAULT NOW(),  created_at      TIMESTAMPTZ DEFAULT
NOW());
```

Table: specialists_on_duty

```
CREATE TABLE specialists_on_duty (  id             UUID PRIMARY KEY DEFAULT
gen_random_uuid(),  hospital_id    UUID REFERENCES hospitals(id) ON DELETE CASCADE,
specialist_type VARCHAR(100) NOT NULL,  --
'neurosurgeon','cardiologist','trauma_surgeon', etc.  doctor_name    VARCHAR(200),
on_duty        BOOLEAN DEFAULT TRUE,  shift_ends_at  TIMESTAMPTZ,  updated_at
TIMESTAMPTZ DEFAULT NOW());
```

Table: ambulances

```
CREATE TABLE ambulances (  id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
call_sign      VARCHAR(50) UNIQUE NOT NULL,  -- e.g. 'AMB-007'  driver_name
VARCHAR(200),  driver_phone   VARCHAR(20),  current_lat    DECIMAL(10,7),  current_lng
DECIMAL(10,7),  status         VARCHAR(30) DEFAULT 'idle',                 -- 'idle' |
'en_route_to_scene' | 'on_scene' | 'en_route_to_hospital' | 'at_hospital'
assigned_hospital_id UUID REFERENCES hospitals(id),  active_patient_id    UUID,
last_location_update TIMESTAMPTZ DEFAULT NOW(),  created_at     TIMESTAMPTZ DEFAULT
NOW());
```

Table: patients

```
CREATE TABLE patients (  id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
session_code   VARCHAR(20) UNIQUE NOT NULL,  -- short human-readable code, e.g.
'P-20241031-001'  ambulance_id   UUID REFERENCES ambulances(id),  age            INT,
sex            VARCHAR(10),  -- 'male' | 'female' | 'unknown'  chief_complaint TEXT,
mechanism_of_injury TEXT,  -- Latest vitals snapshot (updated in real-time from
vitals_stream)  heart_rate     INT,  systolic_bp    INT,  diastolic_bp   INT,  spo2
DECIMAL(5,2),  respiratory_rate INT,  temperature    DECIMAL(5,2),  gcs_score
INT,          -- Glasgow Coma Scale 3-15  -- ML outputs  predicted_severity INT,
-- 1 (minor) to 5 (critical)  predicted_care_needs JSONB,  -- { "icu": true,
"ventilator": false, "specialist": "trauma_surgeon" }  survivability_score
DECIMAL(5,2),  -- 0.0-1.0  -- Routing  assigned_hospital_id UUID REFERENCES
hospitals(id),  routing_rationale JSONB,  status         VARCHAR(30) DEFAULT 'active',
created_at     TIMESTAMPTZ DEFAULT NOW(),  resolved_at    TIMESTAMPTZ);
```

Table: vitals_stream

```
CREATE TABLE vitals_stream (  id             UUID PRIMARY KEY DEFAULT
gen_random_uuid(),  patient_id     UUID REFERENCES patients(id) ON DELETE CASCADE,
source         VARCHAR(30) DEFAULT 'monitor',  -- 'monitor' | 'manual'  heart_rate
INT,  systolic_bp    INT,  diastolic_bp   INT,  spo2           DECIMAL(5,2),
respiratory_rate INT,  temperature    DECIMAL(5,2),  gcs_score      INT,  recorded_at
TIMESTAMPTZ DEFAULT NOW());CREATE INDEX idx_vitals_patient ON
vitals_stream(patient_id, recorded_at DESC);
```

Table: routing_events

```
CREATE TABLE routing_events (  id             UUID PRIMARY KEY DEFAULT
gen_random_uuid(),  patient_id     UUID REFERENCES patients(id),  hospital_id    UUID
REFERENCES hospitals(id),  triggered_at   TIMESTAMPTZ DEFAULT NOW(),  transit_time_min
INT,  survivability_score DECIMAL(5,2),  shap_explanation JSONB,  -- per-feature
importance values  all_hospitals_scored JSONB,  -- ranked list of all evaluated
hospitals  override_by_dispatcher BOOLEAN DEFAULT FALSE,  event_type     VARCHAR(20)
DEFAULT 'single'  -- 'single' | 'mce_batch');
```

Table: mce_events

```
CREATE TABLE mce_events (  id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
triggered_at   TIMESTAMPTZ DEFAULT NOW(),  patient_ids    UUID[],  batch_assignments
JSONB,  -- [{ patient_id, hospital_id, score }]  resolved_at    TIMESTAMPTZ,  notes
TEXT);
```

1.2 — Seed Data

Seed the database with 10 simulated hospitals representing a regional grid (use realistic Indian city coordinates — e.g. Mumbai/Pune region). Vary capabilities so the routing engine has real choices to make:

- 2 hospitals with full trauma centres, cardiac surgery, neurosurgery (Level 1 equivalent)
- 3 hospitals with trauma centres but limited specialist availability
- 3 general hospitals with good bed availability but no specialist surgery
- 2 hospitals intentionally seeded as near-capacity (90%+ load) to test load-balancing

- Seed 3 ambulances with call signs AMB-001, AMB-002, AMB-003
- Seed 1 active patient with vitals in vitals_stream (for demo/testing)

STAGE 2 — Backend API — Core Endpoints

2.1 — Node.js / Express API

Base URL: http://localhost:3001/api

Hospital Endpoints

METHOD	ENDPOINT	PURPOSE	REQUEST BODY / PARAMS
GET	/hospitals	List all active hospitals with full resource state	Query: ?lat=&lng=&radius_km= (optional filter)
GET	/hospitals/:id	Single hospital with full detail including specialists on duty	Path param: id
PUT	/hospitals/:id/resources	ERC staff updates bed counts, equipment availability	{ avail_icu_beds, avail_gen_beds, avail_ventilators, current_load_pct }
PUT	/hospitals/:id/specialists	ERC staff updates specialist on-duty status	{ specialist_type, on_duty, shift_ends_at }
GET	/hospitals/:id/incoming	List all ambulances currently routed to this hospital	—

Patient & Vitals Endpoints

METHOD	ENDPOINT	PURPOSE	REQUEST BODY
POST	/patients	Create new patient session (called when AMB picks up patient)	{ ambulance_id, age, sex, chief_complaint, mechanism_of_injury }
GET	/patients/:id	Full patient record with latest vitals and routing assignment	—
POST	/patients/:id/vitals	Push vitals reading (called by patient monitor device every 15s)	{ heart_rate, systolic_bp, diastolic_bp, spo2, respiratory_rate, temperature, gcs_score, source }

GET	/patients/:id/vitals/history	Retrieve vitals time-series for sparkline display	Query: ?limit=50
POST	/patients/:id/route	Trigger APEX + NEXUS routing engine for this patient	{ force_reroute: bool }
PUT	/patients/:id/resolve	Mark patient as resolved/transferred	{ outcome }

Ambulance Endpoints

METHOD	ENDPOINT	PURPOSE	REQUEST BODY
GET	/ambulances	List all ambulances with current status and location	—
PUT	/ambulances/:id/location	Driver app pushes GPS location every 10 seconds	{ lat, lng }
PUT	/ambulances/:id/status	Update ambulance operational status	{ status }
GET	/ambulances/:id/active-patient	Get current active patient for this ambulance	—

Routing Engine Endpoint (proxied to FastAPI)

METHOD	ENDPOINT	PURPOSE	REQUEST BODY
POST	/route/single	Single patient routing — calls APEX then NEXUS	{ patient_id } — pulls vitals from DB automatically
POST	/route/mce-batch	Mass casualty batch routing for multiple simultaneous patients	{ patient_ids: [] }
GET	/route/events	History of all routing decisions with SHAP explanations	Query: ?patient_id= or ?hospital_id=

2.2 — WebSocket Events (Socket.io)

The Node.js server is the WebSocket hub. All three frontend interfaces connect to it. Use rooms: hospital:{id}, ambulance:{id}, global.

EVENT NAME	DIRECTION	PAYLOAD	SUBSCRIBERS
vitals:update	Server → Client	{ patient_id, vitals, predicted_severity }	ambulance-driver app, patient-monitor app
routing:decision	Server → Client	{ patient_id, hospital_id, rationale, eta_min, survivability }	All interfaces
hospital:status_update	Server → Client	{ hospital_id, avail_icu_beds, load_pct, incoming_count }	All interfaces
ambulance:location	Client → Server	{ ambulance_id, lat, lng }	Broadcast to hospital-erc and global
ambulance:location_broadcast	Server → Client	{ ambulance_id, lat, lng, status, patient_severity }	hospital-erc, global
mce:triggered	Server → Client	{ event_id, patient_count, assignments[] }	All interfaces — triggers MCE mode UI
routing:reroute	Server → Client	{ patient_id, new_hospital_id, reason }	ambulance-driver app

STAGE 3 — ML Engine — APEX Severity Predictor (FastAPI)

3.1 — APEX: What It Does

APEX takes raw patient vitals and metadata as input and outputs two things: (1) predicted critical care requirements (ICU needed, ventilator needed, specialist type), and (2) a survivability score (0.0–1.0) representing the probability of good outcome given optimal routing.

3.2 — Synthetic Training Data

Since real patient data is unavailable for development, generate synthetic training data using clinical logic. Create a Python script generate_training_data.py that produces 10,000 rows following these rules:

- GCS < 9 → likely needs ICU + possibly ventilator
- SpO2 < 90% → likely needs ventilator
- Systolic BP < 80 → critical, ICU required
- HR > 140 or < 40 → critical
- Chief complaint = 'chest pain' + male + age > 50 → likely needs cardiologist + cath lab
- Chief complaint = 'head injury' + GCS < 12 → likely needs neurosurgeon
- Chief complaint = 'trauma' + BP < 90 → likely needs trauma surgeon
- Chief complaint = 'burns' → needs burn unit

Add realistic Gaussian noise to vitals. Save as training_data.csv.

3.3 — Model Training Script (train_apex.py)

Train an XGBoost classifier. Target labels: { needs_icu, needs_ventilator, specialist_required (multi-class) }. Steps:

1. Load training_data.csv
2. Feature engineering: pulse_pressure = systolic_bp - diastolic_bp; shock_index = hr / systolic_bp
3. Train separate XGBoost binary classifiers for needs_icu and needs_ventilator
4. Train XGBoost multi-class classifier for specialist_required (classes: none, trauma_surgeon, cardiologist, neurosurgeon, paediatric, burn_specialist)
5. Compute SHAP values using shap.TreeExplainer — save explainer object
6. Calculate survivability_score as a weighted composite: 0.4×(1-severity_normalised) + 0.3×gcs_normalised + 0.3×spo2_normalised
7. Save models as apex_icu.pkl, apex_ventilator.pkl, apex_specialist.pkl, shap_explainer.pkl

3.4 — FastAPI Prediction Endpoint

POST /ml/predict

```
Input: { patient_id, heart_rate, systolic_bp, diastolic_bp, spo2, respiratory_rate,
temperature, gcs_score, age, sex, chief_complaint, mechanism_of_injury }
```

```
Output: { needs_icu, needs_ventilator, specialist_required, predicted_severity (1-5),
survivability_score, shap_values: { feature: importance } }
```

After returning prediction, automatically write results to patient record in PostgreSQL.

STAGE 4 — Constraint Optimizer — NEXUS Routing Engine

4.1 — NEXUS: What It Does

NEXUS is an integer linear programming optimizer. It receives APEX's output (what the patient needs) and the live hospital grid (what each hospital has), and computes the optimal hospital assignment. It must be fast — target under 500ms for single patient, under 2 seconds for 20-patient MCE batch.

4.2 — Single Patient Routing

POST /ml/route — Input: { patient_id, apex_output, hospitals_list }

Algorithm:

8. Fetch all active hospitals from PostgreSQL
9. Hard constraint elimination: remove any hospital missing a required resource (e.g. needs ventilator → remove hospitals with 0 ventilators, needs neurosurgeon → remove hospitals without neurosurgery + on-duty neurosurgeon, needs ICU → remove hospitals with 0 avail ICU beds)
10. Score remaining hospitals using weighted formula:
    - transit_score = (1 - transit_time / max_transit_time) × 40 ← 40% weight
    - survivability_boost = survivability_score × 30 ← 30% weight
    - load_score = (1 - current_load_pct/100) × 20 ← 20% weight
    - resource_richness = (matching_resources / needed_resources) × 10 ← 10% weight
11. Return ranked list of hospitals with scores, plus the top recommendation
12. Write routing decision to routing_events table
13. Emit routing:decision WebSocket event
14. Calculate transit time using Google Maps Distance Matrix API (or OSRM fallback)

4.3 — MCE Batch Routing (Multi-Patient)

POST /ml/mce-route — Input: { patient_ids: [] }

Use PuLP integer linear programming to assign N patients to M hospitals simultaneously:

- Objective: Maximise total_survivability_weighted_by_score across all assignments
- Hard constraint: No hospital can receive more patients than its available ICU/gen beds
- Hard constraint: Each patient must be assigned to exactly one hospital
- Hard constraint: A hospital without required specialist cannot receive a patient needing that specialist
- Soft constraint: Penalise assignments that concentrate >40% of patients at a single hospital

After solving, emit mce:triggered WebSocket event with all assignments. Write to mce_events table.

STAGE 5 — Interface A — Hospital Emergency Response Centre (ERC) Dashboard

USER

Hospital ERC staff — seated at large desktop monitor, managing incoming ambulances, updating hospital resource status

5.1 — Layout Overview

Full-screen dashboard layout. No mobile required. Target: 1920×1080. Use a dark-accented professional colour scheme (deep navy/dark header with white content areas). The layout is a 3-panel design:

PANEL	WIDTH	CONTENT
Left Sidebar	320px fixed	Hospital own resource management — bed counts, equipment toggles, specialist on-duty toggles
Centre Main	Remaining width	Google Map (60% of centre height) + Incoming Ambulance Queue below map
Right Sidebar	380px fixed	Incoming patient detail panel — shows active patient brief, vitals summary, routing decision details

5.2 — Top Header Bar

- Hospital name (large, bold) + current time (live clock)
- 3 stat pills: AVAILABLE ICU BEDS (green/amber/red based on count) | AVAILABLE GEN BEDS | CURRENT LOAD %
- Alert bell icon — badge count of unread routing notifications
- MCE Mode toggle button — glows red when active, arms the batch routing display
- Status indicator: ONLINE (green pulsing dot) — WebSocket connection status

5.3 — Left Sidebar: Resource Management Panel

This panel is the primary tool for ERC staff to keep hospital data accurate.

Bed Management Section

- ICU Beds: stepper input (−/+) for available count, total shown as denominator (e.g. '4 / 12')
- General Beds: same stepper input pattern
- Each stepper auto-saves to PUT /hospitals/:id/resources on change — no save button needed
- Colour coding: ≥50% = green, 20-49% = amber, <20% = red

Equipment Section

- Ventilators: stepper for available count (0 to total)

- Cath Lab: toggle switch — Available / Occupied
- OR Theatres: stepper for available count

Specialists On-Duty Section

For each specialist type relevant to this hospital (trauma surgeon, cardiologist, neurosurgeon, paediatric surgeon, burn specialist, anaesthesiologist):

- Toggle row: Specialist name + type + On Duty / Off Duty toggle
- When toggled ON: show shift end time picker
- Changes auto-save to PUT /hospitals/:id/specialists
- On-duty specialists shown in green, off-duty in grey

Quick Actions

- ACCEPT DIVERSION button — manually mark hospital as accepting diverted ambulances
- DECLARE CAPACITY button — marks hospital at capacity, stops new routing assignments

5.4 — Centre Panel: Google Map

Map height: 55% of viewport height. Full width of centre panel.

Google Maps JS API implementation:

- Map centred on hospital's own coordinates, zoom level 12
- Own hospital marker: large red cross icon (hospital pin)
- Other hospitals in region: smaller grey hospital pins
- Active ambulances en route TO this hospital: pulsing ambulance icons in red
- Active ambulances en route to OTHER hospitals: smaller grey ambulance icons
- Route polyline: draw Directions API route from each incoming ambulance to this hospital in red dashed line
- ETA label: floating label on each route line showing minutes remaining
- Click on ambulance marker → opens right sidebar with that patient's details
- Map auto-pans to fit all incoming ambulances in view
- Live location updates via WebSocket — ambulance markers move in real time

5.5 — Incoming Ambulance Queue (Below Map)

Horizontal scrollable card row. Each card represents one incoming ambulance:

CARD ELEMENT	CONTENT
Header	Ambulance call sign (e.g. AMB-007) + colour-coded severity badge (CRITICAL / HIGH / MODERATE)
ETA	Large countdown timer showing minutes:seconds to arrival — counting down live

Patient Summary	Age, sex, chief complaint, mechanism of injury
Care Needs	Icon row: ICU icon (if needed), Ventilator icon, Specialist badge (e.g. 'Trauma Surgeon')
Vitals Strip	Small inline sparklines for HR and SpO2 over last 5 minutes
Survivability	Circular progress indicator 0-100% — colour: red <40%, amber 40-70%, green >70%
Actions	PREPARE TEAM button → marks team as activated; VIEW DETAILS → expands right sidebar

5.6 — Right Sidebar: Patient Detail Panel

Appears when a card or map ambulance marker is clicked. Contains:

- Patient session code + ambulance call sign
- Full vitals grid: HR, BP, SpO2, RR, Temp, GCS — each with trend arrow (↑/↓/→)
- Vitals sparkline charts for HR and SpO2 (last 10 readings from vitals history API)
- APEX Prediction box: Predicted Severity (1-5 scale with visual bar), Care Requirements list, Survivability Score
- Routing Rationale box (SHAP panel, simplified): 'Routed to YOUR HOSPITAL because: [top 3 reasons as plain-English sentences]'
- ETA Countdown (large, prominent)
- PRE-ARRIVAL CHECKLIST: expandable checklist of prep steps based on care needs (e.g. 'Reserve ICU Bed 4', 'Alert Trauma Surgeon Dr. Sharma', 'Activate Trauma Bay 2')
- REROUTE button (red) — triggers dispatcher override flow

5.7 — MCE Mode View

When mce:triggered event received (or when MCE Mode button is activated):

- Dashboard background flashes amber border twice, then locks into MCE layout
- Centre panel splits: LEFT half shows incoming patient list (all active MCE patients as cards), RIGHT half shows hospital assignment matrix
- Assignment matrix: grid of patient rows × hospital columns with assignment lines drawn
- Each cell shows: match score, or 'ASSIGNED' badge, or 'FULL' or 'MISSING RESOURCE' in red
- Dispatcher can drag a patient card to a different hospital column to override NEXUS — system recalculates all other assignments
- MCE event timer in top bar showing elapsed time since MCE triggered

STAGE 6 — Interface B — Ambulance Driver App

USER

Ambulance driver — device is a tablet mounted on dashboard. Gloves may be worn. One-hand operation required. High-contrast UI for bright conditions.

6.1 — Layout Overview

Single-page app, mobile-first (tablet 768px – 1024px). Portrait orientation. The UI is divided vertically:

- Top 60% of screen: Google Map with navigation overlay
- Bottom 40% of screen: Hospital destination card + patient summary strip

No sidebars. Maximum clarity. Minimum taps required.

6.2 — Ambulance Login / Session Start

- Simple login screen: Select ambulance call sign from dropdown (AMB-001 / AMB-002 / AMB-003)
- Enter PIN (4-digit, simple auth for demo)
- On login: start GPS location push loop (every 10 seconds → PUT /ambulances/:id/location)
- App stays awake — prevent screen sleep

6.3 — Status Flow Screens

The app has distinct screens for each ambulance status, switching automatically:

STATUS	SCREEN SHOWN
idle	STANDBY screen — large green 'READY' indicator, ambulance call sign, current time. Single button: 'PICK UP PATIENT'
on_scene	PATIENT INTAKE screen — form to enter patient details (see 6.4). Status automatically advances when patient record created.
en_route_to_hospital	NAVIGATION screen — full map with route + destination card (see 6.5). This is the primary operational screen.
at_hospital	HANDOVER screen — shows 'ARRIVED AT [HOSPITAL NAME]'. Button: 'CONFIRM HANDOVER'. Triggers patient resolve.

6.4 — Patient Intake Screen (On Scene)

Keep this screen minimal — the driver or EMT needs to enter data fast while managing a patient.

- Patient Age: number input (large, big tap target)

- Patient Sex: 3 large toggle buttons — Male / Female / Unknown
- Chief Complaint: scrollable list of common complaints as large tile buttons (Chest Pain / Head Injury / Trauma / Difficulty Breathing / Burns / Stroke / Cardiac Arrest / Other)
- Mechanism of Injury: similar tile buttons (RTA / Fall / Assault / Medical / Industrial / Other)
- Manual vitals entry (OPTIONAL — auto-filled if Patient Monitor is connected): HR, BP, SpO2, GCS — all numeric inputs
- Large green 'GET ROUTING' button — creates patient record (POST /patients), pushes vitals (POST /patients/:id/vitals), then triggers routing (POST /route/single)
- While routing runs: show a full-screen 'FINDING BEST HOSPITAL...' loading state with animated pulse

6.5 — Navigation Screen (En Route)

This is the most important screen. Design for maximum legibility at a glance while driving.

Top 60% — Google Map Navigation

- Google Maps JS API embed with real-time Directions API route drawn
- Map in 'driving' mode — route highlighted with thick blue polyline
- Destination hospital marked with large red cross pin
- Current ambulance position (GPS) shown as pulsing blue arrow
- Turn-by-turn navigation: next turn instruction shown as a floating overlay banner at top of map — 'In 200m, TURN RIGHT onto Linking Road'
- Route recalculates automatically if driver deviates — call Directions API again on position update
- Traffic layer enabled — show real-time traffic delays on route

Bottom 40% — Destination & Patient Strip

Divided into two rows:

ROW 1 — Hospital Destination Card:

- Hospital name (large font, 24px+)
- ETA pill — '12 MIN' in large bold — updates live as GPS updates
- Distance remaining — e.g. '4.2 km'
- Hospital readiness status: 'TEAM NOTIFIED' (green) / 'PREPARING' (amber) / 'NOT YET NOTIFIED' (grey)
- Change Destination button (small, grey) — for dispatcher override

ROW 2 — Patient Summary Strip:

- Severity badge (CRITICAL / HIGH / MODERATE / LOW) — colour coded
- 3 key vitals inline: ❤️ HR=110 | 🩸 BP=90/60 | □ SpO2=94% — updates in real time if monitor connected
- Care needs icons: ICU / Ventilator / Trauma Surgeon (as small coloured badges)
- Survivability score: circular dial, colour-coded

6.6 — Rerouting Alert

- If routing:reroute WebSocket event is received: full-screen overlay modal with loud alert
- Shows: 'ROUTE CHANGED — New destination: [Hospital Name]' in large text
- Reason: e.g. 'City General reached capacity. Rerouted to Apollo (2 min longer, better match)'
- Two buttons: ACCEPT NEW ROUTE (green, large) | KEEP CURRENT (red, smaller)
- Google Map automatically updates route when new destination accepted

STAGE 7 — Interface C — Patient Monitor Device UI

USER

Autonomous / semi-autonomous. This is an embedded touchscreen UI on the ambulance's health monitoring device. It reads vitals from connected hardware sensors and pushes to backend. A paramedic may also interact with it.

7.1 — Concept

This interface simulates an embedded medical device display — like the screen on a Zoll or Philips patient monitor, but connected to AURUM. In real deployment, this would run on dedicated hardware. For the project, build a fullscreen web app designed for an 800×480 landscape touchscreen.

The device has two purposes:

- Primary: Continuously stream patient vitals to the AURUM backend (POST /patients/:id/vitals every 15 seconds)
- Secondary: Display the routing recommendation, ETA, and current vitals so the paramedic treating the patient can see it without picking up a phone

7.2 — Layout (800×480 Landscape)

Use a dark theme — dark charcoal background (#0F1117) with bright coloured readouts. This matches real medical device aesthetics.

ZONE	SIZE	CONTENT
Left — Vitals Panel	50% width	Live vitals readouts (see 7.3)
Right Top — Routing Panel	50% width, 60% height	Hospital routing result + ETA + map thumbnail (see 7.4)
Right Bottom — Status	50% width, 40% height	Device status, connection state, session info (see 7.5)

7.3 — Left Panel: Live Vitals Readouts

Display these vitals as large, prominent numeric readouts in the style of a real medical monitor:

- Heart Rate: large yellow number (e.g. '110') + 'bpm' in smaller text + realtime ECG-style waveform animation below number
- Blood Pressure: large white number '90 / 60' + 'mmHg' — systolic/diastolic

- SpO2: large cyan number '94' + '%' — with small pleth waveform
- Respiratory Rate: white number '22' + '/min'
- Temperature: white number '37.8' + '°C'
- GCS: large number with sub-scores (E4 V3 M5 = 12) — colour coded green/amber/red

Critical alert logic: if any vital crosses danger threshold (e.g. SpO2 < 90, HR > 150, GCS < 8, BP < 80 systolic), that readout flashes red and an alarm sound fires (use Web Audio API).

7.4 — Right Top Panel: Routing Result

- Shows result of latest routing decision from WebSocket event routing:decision
- Hospital name in large bold white text
- ETA in large coloured countdown timer
- Survivability score circular indicator
- Care requirements icons (ICU / Ventilator / Specialist badge)
- Small embedded Google Map (300×200px) showing route from current GPS to hospital — static map or embedded JS map. Refresh every 30 seconds.
- Hospital readiness badge: 'TEAM NOTIFIED' / 'PREPARING'
- If no routing decision yet: show 'AWAITING ROUTING...' with animated pulse

7.5 — Right Bottom Panel: Device Status

- Patient session code (e.g. P-20241031-001) — tap to change session
- Ambulance call sign (e.g. AMB-007)
- Connection status: CONNECTED to AURUM (green pulsing dot) / OFFLINE (amber)
- Last vitals push timestamp: 'Last sync: 14:32:05'
- Data source indicator: SENSORS (if simulating hardware input) / MANUAL
- PUSH NOW button: immediately sends current vitals (for out-of-cycle manual trigger)

7.6 — Vitals Input (Simulation Mode)

Since real hardware sensors won't be available in development, build a simulation mode:

- On device startup: show a setup screen to either CONNECT TO SESSION (enter patient session code) or START NEW SESSION
- In simulation mode: vitals auto-increment/decrement with realistic noise around seeded baseline values
- Paramedic can tap any vital readout to manually adjust it — numeric keypad appears
- Every 15 seconds: auto-POST current vitals to /patients/:id/vitals with source: 'monitor'
- On any critical vitals change (threshold breach): immediately POST vitals + trigger routing re-evaluation via POST /patients/:id/route

STAGE 8 — Real-Time Layer — WebSockets & Live Updates

8.1 — Socket.io Server Setup

Configure the Node.js server with Socket.io. Use Redis adapter (socket.io-redis) so events can be published from the FastAPI service via Redis pub/sub and consumed by the Socket.io hub.

- On client connect: client sends { role: 'hospital_erc' | 'ambulance_driver' | 'patient_monitor', entity_id: '...' }
- Server joins client to appropriate rooms: role-based rooms + entity-specific room
- Ambulance driver joins room: ambulance:{id}
- Hospital ERC joins room: hospital:{id} + global_dispatch
- Patient monitor joins room: ambulance:{id} + patient:{id}

8.2 — Auto-Routing Trigger

When a new vitals push arrives at POST /patients/:id/vitals:

15. Save vitals to vitals_stream table
16. Update patient vitals snapshot in patients table
17. If this is the FIRST vitals push for this patient session: automatically call POST /route/single
18. If vitals indicate a critical change (SpO2 drops below 90%, GCS drops by 2+ points): automatically re-trigger routing
19. Emit vitals:update event to ambulance:{id} and patient:{id} rooms

8.3 — Hospital Status Change Propagation

When ERC staff updates hospital resources via PUT /hospitals/:id/resources:

- Save to database
- Emit hospital:status_update to global_dispatch room
- If the change reduces availability (e.g. ICU beds drop to 0): check all currently routed patients who were assigned to this hospital and trigger re-routing for any with critical needs

STAGE 9 — Google Maps Integration — All Three Interfaces

KEY

Use the Google Maps JavaScript API for interactive maps. Use the Directions API for route drawing. Use the Distance Matrix API for ETA/distance calculation in the routing engine. Store your API key in .env as VITE_GOOGLE_MAPS_API_KEY (frontend) and GOOGLE_MAPS_API_KEY (backend).

9.1 — Shared Map Component

Build a reusable React component <AurumMap> that accepts props and handles all three use cases. Place in frontend/src/components/shared/AurumMap.jsx.

PROP	TYPE	PURPOSE
center	{ lat, lng }	Map centre coordinates
zoom	number	Initial zoom level
hospitals	Hospital[]	Array of hospital objects to render as pins
ambulances	Ambulance[]	Array of active ambulances with location
routes	RouteConfig[]	Array of { origin, destination, color } to draw as polylines
onMarkerClick	function	Callback when a marker is clicked
showTraffic	boolean	Enable traffic layer
height	string	CSS height value
mode	string	'erc' | 'driver' | 'monitor' — affects UI overlays

9.2 — Per-Interface Map Configuration

Interface A — Hospital ERC Map

- Height: 55vh
- Shows: own hospital (large pin) + all regional hospitals + all active ambulances
- Routes: Directions API routes from each incoming ambulance to this hospital (dashed red polyline)
- Ambulance markers update position in real time via ambulance:location_broadcast WebSocket event
- Map bounds auto-fit to include all incoming ambulances
- Hospital heat overlay: change marker colour based on load % (green ≥50% capacity free, amber 20-50%, red <20%)

Interface B — Ambulance Driver Map

- Height: 60vh
- Shows: current ambulance GPS position + destination hospital pin only
- Routes: single Directions API route from current position to assigned hospital (thick blue polyline)
- Turn-by-turn: parse Directions API steps array, show next step as floating overlay
- Live tracking: re-call Directions API every 30 seconds to update route with live traffic
- Focus: minimal, distraction-free — only destination and route visible

Interface C — Patient Monitor Map

- Height: 200px (compact thumbnail)
- Shows: ambulance location + destination hospital only
- Route drawn as simple polyline (no turn-by-turn needed)
- Static or low-refresh (every 60 seconds) — this is a reference view, not navigation
- No interaction required — display only

9.3 — Distance Matrix API (Backend — NEXUS)

In the NEXUS routing engine (Python FastAPI), use the Google Maps Distance Matrix API to get accurate transit times for each candidate hospital:

- POST /ml/route → before scoring hospitals, call Distance Matrix API with: origins=[ambulance_current_location], destinations=[all_candidate_hospital_locations], mode=driving, departure_time=now (for traffic-aware ETA)
- Use returned duration_in_traffic_value (seconds) as transit_time in NEXUS scoring formula
- Cache results for 60 seconds to avoid redundant API calls during MCE batch routing
- Fallback: if API call fails or key not set, use Haversine distance ÷ 40 km/h as estimated transit time

STAGE 10 — Mass Casualty Event (MCE) Batch Mode

10.1 — MCE Trigger

MCE mode is triggered either:

- Manually by dispatcher clicking MCE MODE button in Hospital ERC Interface
- Automatically when 3 or more simultaneous active unrouted patients exist in the system

When triggered: server creates an mce_events record, broadcasts mce:triggered event to all clients, and calls the MCE batch routing endpoint.

10.2 — MCE Batch Routing Behaviour

The MCE router uses PuLP ILP (see Stage 4.3). Additional rules:

- No single hospital can receive more than 40% of total MCE patients (surge prevention)
- If a patient has critical severity (5), they are treated as a priority constraint — their assignment is optimised first
- Output: array of { patient_id, hospital_id, score, rationale } — one per patient
- Write complete batch to mce_events.batch_assignments
- Emit mce:triggered WebSocket event with full assignments

10.3 — MCE UI Behaviour

All three interfaces respond to mce:triggered:

- Hospital ERC: switch to MCE Grid Allocation view (described in Stage 5.7)
- Ambulance Driver: if driver's patient is being reassigned, show rerouting alert (Stage 6.6)
- Patient Monitor: routing panel updates automatically with new hospital assignment

STAGE 11 — Explainability Panel — SHAP Routing Rationale

11.1 — What to Build

Every routing decision must come with a human-readable explanation. This is a core requirement of the problem statement. The explainability panel appears in the Hospital ERC Interface (right sidebar) and in a compact version in the Ambulance Driver app (tap to expand).

11.2 — Plain-English Sentences

Convert SHAP values into plain English. Map each top SHAP feature to a sentence template:

- needs_icu = true + icu_available → 'Requires ICU — this hospital has 4 ICU beds available'
- specialist_required = neurosurgeon → '[Hospital] has Dr. Mehta (Neurosurgeon) on duty'
- low_load_score for competitor → 'City General excluded — currently at 94% capacity'
- transit_time wins → '3 minutes faster than next nearest suitable hospital'
- survivability boost → 'Survivability score 12% higher with this hospital's resources'

Show top 3 reasons as bullet points in plain English. Below that, show a horizontal bar chart of all SHAP feature values (technical view — toggleable).

11.3 — Full Ranked Hospital List

In the Hospital ERC right sidebar, below the plain-English rationale, show a ranked table of ALL hospitals evaluated:

- Hospital name | Total Score | Missing Resources (if excluded) | Transit Time | Load %
- Selected hospital highlighted in green
- Excluded hospitals (hard constraint failures) shown in red with reason
- Other candidates shown ranked by score

—— BONUS FEATURES ——

Complete all STAGE 0–11 items before starting any BONUS feature.

STAGE A — ORACLE — Predictive Hospital Capacity (30-Minute Forward Projection)

Concept

Instead of routing to the hospital that has capacity NOW, route to the hospital that will have capacity WHEN THE AMBULANCE ARRIVES. ORACLE maintains a rolling 30-minute projection of each hospital's state.

Implementation

- Add an oracle_projections table: { hospital_id, projected_at, avail_icu_projected, avail_gen_projected, confidence_pct }
- Build a background job (cron, runs every 5 minutes) that projects each hospital's capacity using: current_avail_beds - incoming_ambulances_eta_within_30min + expected_discharges_in_30min (estimated as 2% of occupied beds per 30min as a baseline heuristic)
- NEXUS routing engine: add a flag use_oracle_projection=true that uses projected values instead of current values when calculating scores
- Hospital ERC Map: add a timeline scrubber control below the map — dragging it from NOW to +30min animates hospital pin colours changing based on ORACLE projections
- ERC Sidebar: add a PROJECTED CAPACITY section showing current vs projected values for this hospital

STAGE B — Voice Input — Whisper ASR for EMT (Patient Intake)

Concept

Allow the EMT to speak patient vitals aloud instead of typing. The voice input processes audio in-browser using the Web Speech API (for demo) or an optional Whisper API call.

- Add a microphone button to the Patient Intake screen (Interface B)
- When pressed: start Web Speech API recording. Show live transcript on screen
- Parser: regex-based NLP parser that extracts vitals from spoken text: 'BP one forty over ninety' → { systolic_bp: 140, diastolic_bp: 90 }, 'heart rate one ten' → { heart_rate: 110 }, 'GCS fourteen' → { gcs_score: 14 }
- Parsed values auto-fill the intake form fields — EMT can review and correct
- Show confidence indicator for each parsed field (green = confident parse, amber = uncertain)

- Fallback: if parse fails for any field, that field stays blank and highlights for manual entry

STAGE C — HELIX — Post-Incident Analytics Dashboard

Concept

A separate analytics view (accessible from Hospital ERC, gated behind an 'Analytics' tab) showing system performance over time.

- Chart 1: Routing decisions over time — line chart of total routing events per hour
- Chart 2: Average survivability score by hospital — bar chart
- Chart 3: Regional capacity heatmap — map overlay showing average load% by hospital over last 24h
- Chart 4: MCE response timeline — timeline showing MCE events, duration, and patient distribution
- Chart 5: Prediction accuracy — (in simulation) compare predicted care needs vs 'actual' (seeded outcomes)
- Use Recharts or Chart.js for all charts
- Export to CSV button for all charts

STAGE D — Pre-Arrival Activation Protocol

Concept

The moment NEXUS selects a hospital, AURUM automatically sends a structured Pre-Arrival Brief to that hospital's ERC interface — activating their trauma team before the ambulance arrives.

- On routing decision: server sends a pre_arrival_brief object to hospital:{id} room via WebSocket
- Brief contains: patient age/sex, severity, care needs, ETA, top 3 vitals, EMT name, ambulance ID
- Hospital ERC: shows full-screen notification overlay: 'INCOMING PATIENT BRIEF — ETA 8 MIN' with all brief details
- Auto-generated checklist: based on care_needs, generate a prep checklist. E.g. needs ICU + ventilator → ['Reserve ICU Bed', 'Verify Ventilator Operational', 'Alert Respiratory Therapist', 'Activate Trauma Bay']
- ERC staff can tick off checklist items — status feeds back to ambulance driver app as 'TEAM NOTIFIED' indicator

STAGE E — Federated Learning Architecture Stub

Concept

Demonstrate (as a proof-of-concept stub) that AURUM can improve its APEX prediction model using outcome data without hospitals sharing raw patient records — using a federated learning architecture.

- Create a POST /ml/outcome endpoint — called when a patient is resolved. Accepts: { patient_id, actual_care_used, outcome }
- Store outcome data in a new patient_outcomes table
- Build a stub federated_update.py script that simulates gradient aggregation: loads local outcome data, computes XGBoost gradient updates (not the full data), and 'sends' them to a central aggregator endpoint
- Central aggregator (stub): POST /ml/federated-update — receives gradient updates from hospitals, applies them to the central APEX model
- In Hospital ERC Interface: add a 'Model Contribution' panel (Analytics tab) showing: 'X outcomes contributed this week. Your data improved routing accuracy by Y%.'
- This stub demonstrates the architecture is privacy-preserving — raw patient data never leaves the hospital system

Final Build Checklist — Core Features

Use this checklist to verify the system before demo. All items must be checked before presenting.

Backend & Engine

- PostgreSQL database with all 7 tables created and seeded
- Node.js Express API with all endpoints responding correctly
- FastAPI ML service running with APEX and NEXUS operational
- Socket.io WebSocket hub connected with correct rooms
- Google Maps Distance Matrix API integrated in NEXUS

Interface A — Hospital ERC

- Resource management panel: bed/equipment/specialist updates save in real time
- Google Map showing ambulances en route with live position updates
- Incoming ambulance queue with ETA countdowns
- Patient detail sidebar with SHAP explainability panel
- MCE mode view activates on mce:triggered event

Interface B — Ambulance Driver

- Patient intake form with all fields and 'GET ROUTING' flow
- Navigation screen with Google Maps Directions API route drawn
- Turn-by-turn instruction overlay
- Hospital destination card with live ETA countdown
- Rerouting alert modal on routing:reroute event

Interface C — Patient Monitor

- Vitals readouts displayed in medical-device style
- Simulation mode generating realistic vital sign fluctuations
- Auto-push to POST /patients/:id/vitals every 15 seconds
- Routing result panel updating from WebSocket
- Critical vitals threshold alerts (visual + audio)

Integration

- All three interfaces connected to same backend via WebSocket
- End-to-end flow works: Patient Monitor pushes vitals → APEX runs → NEXUS routes → all UIs update
- MCE scenario: trigger multiple patients → NEXUS batch routes → all UIs switch to MCE mode

"In the golden hour, every second of delay is a decision made by inertia. AURUM turns inertia into intelligence — routing not to where care exists, but to where it will be when it's needed most."

AURUM — Adaptive Unified Routing for Urgent Medicine